Architectural Guardrails: An Opinionated Framework for Preventing Structural Decay in Greenfield Research Software

Art Ó Liatháin
22363092@studentmail.ul.ie
University of Limerick

May 2026

Keywords Static Analysis · Machine Learning · Architectural Technical Debt · Maintability · Research Software · Temporal Coupling · Random Forests · Heuristic Based Dataset Generation

Contents

1 Abstract . . . 4
2 Introduction . . . 5
3 Background Research . . . 7
3.1 The current state of research software . . . 7
3.2 Maintainability . . . 8
3.2.1 What does it mean to be maintainable? . . . 8
3.2.2 Measuring Maintainability . . . 9
3.2.3 Code churn's impact on maintainability . . . 11
3.3 Static Analysis . . . 12
3.3.1 Current Methods . . . 12
3.3.2 Issues with Modern Approaches . . . 12
3.3.3 Rules as Data Approaches . . . 13
3.4 Machine Learning . . . 13
3.4.1 Decision Trees . . . 14
3.4.2 Random Forest . . . 14
3.4.3 Applications to maintainability detection . . . 15
3.5 Summary . . . 15
4 Methodology . . . 17
4.1 Dataset Selection and Acquisition . . . 17
4.1.1 Data collection . . . 17
4.1.2 Dataset First Pass Analysis . . . 19
4.2 Dataset Labeling . . . 20
4.3 Feature Selection. . . . 21

4.3.1 Hub Score ... 21
4.3.2 File classification and oversampling ... 23
4.3.3 Dataset verification method ... 24
4.3.4 Addressing Data Leakage ... 25
4.3.5 Dataset Summary ... 25
4.4 Feature Engineering ... 25
4.5 Model Implementation and Training ... 27
4.6 Rule Derivation Process ... 27
4.7 Evaluation Framework ... 27
4.8 Summary ... 28
5 Results ... 29
5.1 Hub Score Comparison with Codescene ... 29
5.2 Machine Learning Models ... 29
5.2.1 Model Configuration ... 29
5.2.2 Model Performance ... 30
5.2.3 Feature Importance ... 32
5.2.4 Extracted rules ... 33
5.2.5 External Validation ... 34
5.3 Summary of Findings ... 35
5.3.1 Autonomous Labeling Accuracy (RQ1) ... 35
5.3.2 Opinionated Interpretable Guardrail Rule Derivation (RQ2) ... 36
6 Discussion ... 37
6.1 The "High-Pass" Nature of Evolutionary Risk (RQ1) ... 37
6.1.1 Temporal metrics as a Proxy for Structural Decay ... 37
6.1.2 Implications for Large-Scale Research ... 37
6.2 The Structural-Temporal Mismatch (RQ2) ... 37
6.2.1 Confidence in Actionable Guardrails ... 38
6.3 What this means for Research Software ... 38
6.3.1 The Empirical Ground Truth ... 38
6.3.2 The changing landscape from Humans to AI ... 39
6.3.3 Analytical Black Boxes ... 39
6.4 Threats to Validity ... 39
6.4.1 Construct Validity ... 39
6.4.2 Internal Validity ... 40
6.4.3 External Validity ... 40
6.5 Future Work ... 40
7 Conclusion ... 41
Bibliography ... 43

List of Figures

Figure 1 Pre filter Distribution of Collaborators ... 18
Figure 2 Pre filter Distribution of Commits ... 19

Figure 3 Hub Score Distribution across metrics: **a)** Commit Count, **b)** Partner Count, **c)** Code Churn, and **d)** Average Coupling. 22
Figure 4 Hub score distribution across all .cpp samples 24
Figure 5 Distribution of high and low risk files 31
Figure 6 Confusion Matrices of predicted metrics: **a)** Decision Tree SMOTE, **b)** Random Forest SMOTE, ... 32
Figure 7 Confusion Matrices of predicted metrics: **a)** Decision Tree Feature Importance , **b)** Random Forest Feature Importance, ... 33
Figure 8 Decision Tree Visual representation 34
Figure 9 Confusion matrix of Random forest model on ACTS repository 35

List of Tables

Table 1 Extracted code metrics (Ardito *et al.*, 2020) 26
Table 2 High risk identification comparison Codescene vs Hub Score 29
Table 3 Model hyperparameters 30
Table 4 Model Performance Comparison 30

1 Abstract

There is a fundamental misalignment between academic funding cycles and software lifecycles, resulting in "disposable" research code that is neither reproducible nor maintainable. Creating an ecosystem in which research projects lay abandoned after being used, unable to be extended or reused requiring reimplementation for it to be used for further research. Researchers do not have the time or capabilities to learn programming to the degree needed to write truly maintainable code while staying within funding cycles. This study proposes a framework to raise the code quality floor for researchers through an opinionated architectural guardrail for static analysis, a domain specific set of rules for software maintainability. This study employs a bespoke methodology to create domain specific maintainability datasets through historical repository metadata. Leveraging this dataset of over 200 research repositories and 9,000 C++ artifacts to train interpretable machine learning models. This empirical approach shifts the paradigm of code governance from subjective "expert-driven" rules toward data-driven thresholds rooted in the actual evolutionary history of scientific software. By benchmarking our induced rules against industry-standard tools using CodeScene on case studies, we determine whether research software follows unique structural laws that link structural metrics meaningfully to historical technical debt. Our findings suggest that domain specific guardrails could significantly lower the barrier to software sustainability within research, ensuring that scientific artifacts remain in use after the initial grant period concludes allowing researchers to work towards the future rather than reimplementing the past.

2 Introduction

Research software is no longer something that lives on the edge of research.Research software has become the primary infrastructure upon which scientific claims are built, progressing discoveries ranging from the intricate algorithms of genome sequencing to the petabyte-scale data pipelines of the Large Hadron Collider. Software has grown to such an importance in the research software lifecycle its significance cannot be ignored, yet why is software overlooked time and time again?

Buggy software produces incorrect results, incorrect results lead to incorrect conclusions which in turn misinforms future research. This is a pervasive issue in research publishing, as the "publish or perish" model in the current landscape encourages researchers to hack together a solution to get the results as soon as possible. Naturally leading to more bugs being created. A core issue is what comes after, verification is difficult, 90% of researchers relied on learning code through self teaching (Hannay *et al.*, 2009) meaning fixing much less identifying bugs is something outside of the technical scope for many researchers and peer reviewers. Worse still, the social pressure to maintain a "perfect" record can lead to a culture of silence, where identified flaws are buried to avoid the perceived stigma of retraction.

To combat this issue reproducibility should be a cornerstone for allowing research to be verified and trusted. The current state of research software is anything but, (Trisovic *et al.*, 2022) identified that across 9,000 R files only 25% of them could run without error or the need for modification. A key note here is there was no comparison to the original research to ensure results matched only to see if the files could execute without error. This is a shocking result, for the reproducibility to be so low it has long term cascading effects: if code code lacks longevity future researchers will need to re implement previous work to be able to start the next step, slowing future research. It also raises an insidious question:

If researchers produce results using a code artifact that cannot be reproduced, the results they produced may never have existed in the first place.

The research community as a whole has made an effort towards reproducibility in research software (Jiménez *et al.*, 2017; Wilson *et al.*, 2017). Reproducibility is reactive, only concerned if we can run the code now but this study posits that this is a symptom of a deeper unaddressed requirement, Maintainability. Maintainability is proactive ensuring the code not only lives now but ensures the code can be understood, modified and utilised well and efficiently for future research.

Tools like static analysis are imperative to research software as a whole to raise code quality. Allowing automatic analysis to be conducted on large scale repositories catching issues early before they grow to be a larger problem. These tools not only allow for code quality to be understood but they facilitate a targeted approach to refactoring. Unfortunately, in the application to the research domain there is a key issue: Warnings generated by these tools are often ignored due to the high volume generated (Johnson *et al.*,

2013), leading to a situation where without a knowledge of programming to identify fact from fiction potential refactoring gets ignored. Warnings are not enough; there needs to be an opinionated system that uses empirical data to accurately predict if a file will become a maintenance issue before it is added to a larger codebase. Expert-driven rules attempt to generalise across all domains, but each domain has unique patterns and approaches which need to be targeted differently. A tool used for particle movement tracking requires a fundamentally different set of quality metrics than a standard enterprise web application. A general approach ignores the nuance of each domain, a focused set of rules for each domain would reduce the number of false positives increasing the standard of code within each domain.

This study seeks to answer two questions:

- To what extent can historical version control metadata be used to autonomously label architectural risk in domain specific repositories?
- Can opinionated and interpretable rules be derived from structural code metrics to predict these historically-identified risks using machine learning?

To tackle this, an explainable machine learning approach using decision trees and random forests will be used. The primary objective is to leverage these models to derive a foundational set of domain-specific static analysis rules tailored to the structural nuances of research software. The goal is to use machine learning to derive a set of foundational domain specific static analysis rules for research software.

A limitation that has been present previously for machine learning for static analysis is requiring a manually labeled dataset. Datasets such as PROMISE (Boetticher, Menzies and Ostrand, 2007) require a commitment from many developers to accurately label a small sample set, meaning that a domain specific approach would be time consuming and effort intensive. To combat this, an automatic approach using heuristic-based labelling methods will be utilised to quickly curate a high-quantity, domain specific dataset from research repositories sampling from over 200+ different research repositories resulting in a high quality dataset of over 4,000 files. Allowing the ML models to learn patterns specific to research software, increasing the confidence in the resulting quality gate.

Ultimately, this study moves beyond industry-standard domain agnostic static analysis, offering instead a framework of opinionated static analysis guardrails. These guardrails are empirically derived, domain-specific, and built to protect the long-term usability and verifiability of research software. By validating these rules against industry-standard benchmarks,specifically CodeScene. Using unseen case studies from the CERN software ecosystem, this study demonstrates a scalable path toward proactive software maintainability in science

3 Background Research

3.1 The current state of research software

Research software is software made to investigate new ideas and concepts. This comes with one very prominent issue, researchers who are leading the way to new concepts are generally not programmers by trade, 90% of researchers and graduates rely on self teaching in programming (Hannay *et al.*, 2009) Software is made without maintainability in mind only following the check of "It works on my computer". This is best observed by (Trisovic *et al.*, 2022) where over 9000 R files were analysed from the Dataverse Project (http://dataverse.org/). In this study only 25% of R projects were runnable initially, after performing code cleaning, inferring dependencies and altering hardcoded paths 56% were able to be run. This is a shocking finding, as this is irrespective of the code output, only having a successful run of the code was incredibly inconsistent. (Trisovic *et al.*, 2022) did note that journals are beginning to mandate better quality standards for code through research software standards to particularly allow for reproducibility to ensure the results published by the papers are correct. Yet the guidelines for what is needed in terms of quality metrics for the code is still unclear.

Researchers and graduates need clear guidance on how to create and maintain a large software project due to their lack of formal training (Hannay *et al.*, 2009). Unfortunately, mainstream research software guidance remains overwhelmingly focused on reproducibility (Wilson, 2006; Wilson *et al.*, 2017; Jiménez *et al.*, 2017; Marwick, 2017), often relegating other critical facets (such as maintainability and performance) to secondary considerations. While industry utilizes mature tooling and standards to balance reproducibility with code health, academic guidance often fails to provide actionable criteria for software architecture or long-term maintenance. This lack of structural guidance directly contradicts the sustainability goals for research software proposed by (Howison and Herbsleb, 2014). A primary driver of this structural decay is the fundamental mismatch between the nature of software development and the mechanisms of academic funding. Most research grants do not explicitly provision for dedicated Research Software Engineers (RSEs), treating code as a secondary byproduct rather than a core asset requiring professional attention (Goble, 2014). Compounding this issue is that research software is made to fit a funding cycle without a care for the longevity of the codebase. Funding is fundamentally not suited for research software (Howison and Herbsleb, 2014) notes that unlike commercial products where revenue grows linearly with users, research grants are fixed and do not require or reward researchers for user acquisition. Funding pushes research software to focus on itself with no incentives outside of a research paper output. A flawed approach which (Howison and Herbsleb, 2014) does address with two alternative solutions; Commercial models in which the research software itself transitions to a self sufficient project which keeps itself alive and peer-production models which is a similar approach as open-source. Both of these models have merit, they allow research software to live past the funding cycle and continue to improve over time. However, there is one key issue: both rely on the

existence of a maintainable codebase. If a project is built with significant "technical debt" during the funding cycle, the cost of maintenance becomes an insurmountable barrier. A commercial spin-off will fail under the weight of high development costs, and a peer-production community will fail to form because outside contributors cannot navigate or extend a poorly structured codebase. Thus, without a foundation of coding standards, these sustainability models remain aspirational rather than achievable.

Addressing the core issue of research software longevity while adhering to the constraints governing research software as a whole is the gap this paper is seeking to fill. To create an opinionated static analysis tool that will ensure code written has a floor of quality it cannot dip below through objective quality gates. By automating the "quality check" process, the quality of research software maintainability can be raised without requiring researchers to become professional developers. Static analysis provides the immediate, clear guidance that current literature lacks, ensuring that software is not just reproducible today, but maintainable and extensible for the researchers of tomorrow.

3.2 Maintainability

To raise code quality, maintainability must be defined. Maintainability is a term used frequently in software engineering, there is agreed upon definition on what maintainability is but ISO25010 defines it as "The degree of effectiveness and efficiency with which a product or system can be modified to improve it, correct it or adapt it to changes in environment, and in requirements." It defines that the sub-sections of maintainability are *Modularity, Re-usability, Analysability, Modifiability, Testability.* (*ISO 25010*, no date) This is a very broad definition which essentially defines requirements for how easily the system can be modified for change.

The lack of clear testable outcomes for each quality in the ISO standard leads to a conceptual overlap where maintainability becomes a "catch-all" category for non-functional requirements, making assessing what is truly maintainable very difficult. Meaning that while the standards are set out, they are subjective and can change from person to person, organisation to organisation defeating the purpose of standards in the first place. This is why clearly defining what maintainable code looks like is paramount to this paper.

3.2.1 What does it mean to be maintainable?

To define code as maintainable, modularity is one key factor in the assessment. Modularity done correctly facilitates changeability through logical decomposition of functionality, it may seem simple yet if done incorrectly can have long lasting consequences on the maintainability and longevity of a codebase. D.L Parnas presented a study revealing how easily developers could fall into the pitfall of designing the system in a human like manner (Parnas, 1972). Designing systems based on a humans natural flow through a system, converting those actions into a flowchart of logical operations is a method that creates code that is not resilient. While it may be modular on paper as every "step" would be logically separate the code can still be highly coupled through cross cutting dependencies which do

not conform to the human flow, impacting the maintainability of the codebase. An example of this would be for an ordering system receive order -> validate order -> charge card -> reserve stock -> ship order -> send receipt. A flowchart-style design might put all database writes in the "ship order" step because that's when a human thinks the order is complete. But if payment failures, refunds, or partial shipments happen later, every earlier step may need to be revised. In a more robust design, payment, inventory, and fulfillment each own their own data and expose stable interfaces, so changing one rule does not force a rewrite of the whole sequence.

The effect has been substantiated by (Cai *et al.*, 2025) with a direct correlation being made between coupling and maintainability overhead. The core purpose of modularisation should not be encapsulating subroutines into modules. Design decisions that are likely to change should serve as the foundational points to creating modular code, allowing subroutines to be compositions of modules. Taking modularity into account will serve as a solid baseline to assess the maintainability of a codebase and files.

3.2.2 Measuring Maintainability

As maintainability is an abstract concept, various frameworks have attempted to reduce it to a concrete numerical metric. An early approach in this area is the Maintainability Index (MI), which calculates a score based on a weighted combination of Halstead Volume, cyclomatic complexity, lines of code (LOC), and comment density (Oman and Hagemeister, 1992). The formula typically utilized for this assessment is:

$$\mathrm{MI} = 171 - 5.2\ln(V) - 0.23(G) - 16.2\ln(\mathrm{LOC}) + 50\sin\left(\sqrt{2.46 \times C}\right) \tag{1}$$

Where:

- V represents **Halstead Volume**;
- G is **Cyclomatic Complexity**;
- C is the **percentage of comment lines**.

(Oman and Hagemeister, 1992) validated this metric through feedback from 16 developers and empirical testing on industrial systems, including those at Hewlett-Packard and various defense contractors. This methodology provided a pragmatic tool for engineering managers to prioritize maintenance efforts by assigning a tangible value to code quality. While the MI provides a high-level overview of code density, it suffers from what can be described as "semantic blindness". Metrics such as Cyclomatic Complexity and Halstead Volume analyze the control flow and token count of a file but fail to interpret the structural intent or the relationships between components. For the purposes of this paper this lack of overarching analysis will be referred to as "blind metrics". Utilising blind metrics allows for efficient and quick analysis but it opens the door to gaming the system in a sense, where developers could reach extremely maintainable scores syntactically while semantically being unmaintainable, thereby abusing the formula.

For example, A script that maintains a high MI score due to low complexity and short length, yet contain architectural flaws such as "God Objects"(A singular class or file in which does the bulk of the business logic generally being thousands of lines long) or tight

coupling to external datasets that inhibit reuse. The formula was a product of its time: finely tuned to the teams and projects it was applied to, in the modern landscape the current formula would definitely not be accurate with how modern languages have evolved. Even though the formula could be adapted to a modern context it is clear that even a modernised version would suffer from blind metrics and is not something that should be used as only counting lines cannot yield results on the true quality of code.

An approach that would apply in a more modern context was proposed by (Xiao *et al.*, 2016), of identifying architectural debt through evolutionary analysis of a codebase (Xiao *et al.*, 2016). Where architectural debt can be used as a measure of maintainability it quantifies debt through a formalized Debt Model, using regression analysis to capture the growth rate of maintenance costs over time. The paper proposed that there are four key architectural patterns that are the main proponents of ATD, Hub, Anchor Submissive, Anchor Dominant, and Modularity Violation. These patterns are all based on combining structural context alongside evolutionary context to create a holistic view of the codebase.

Hub: Characterised by strong, mutual coupling where the anchor file and its members have structural dependencies in both directions and strong commit coupling in at least one. This pattern often represents "spaghetti code" where a central file is overloaded with responsibilities.

Anchor Submissive: Occurs when members structurally depend on an anchor, but the anchor is historically submissive, changing whenever the members change.This typically indicates an unstable interface being forced to change by its clients.

Anchor Dominant: The reverse of submissive, where members structurally depend on the anchor, and the anchor historically dominates them, frequently propagating changes outward to its dependents.

Modularity Violation: Identified as the most common and expensive form of architectural debt across various projects. It is unique because it involves files that have no explicit structural dependencies but are frequently changed together in the project's commit history.

The four patterns were measured by performing a longitudinal study on the commit histories of large open source codebases. Using tools such as Understand and Titan to derive the relevant pattern by calculating the commit coefficient between every file.

A metric which relates to the chance of a commit on one file will require a commit on another. For example, given files A, B, and C with commit histories of {A,B} and {A,C}:

- If B or C is modified: 100% chance that A will also be modified
- If A is modified: only 50% chance that B or C will be modified

This would then be called a fileset, this would be extrapolated over the entire commit history and codebase creating many filesets. This would allow the tool to extract semantic relations between files that are not apparent structurally. The study was able to compound this effect by measuring the number of commits labeled as bug fixes against the number of

feature commits, this allows simple data analytics to measure the amount of maintenance debt that each fileset would have. Using this method, high maintenance filesets can be labelled and evaluated as a quantitative metric. The core value of their work is the identification of Modularity Violations: instances where files are forced to change in tandem despite having no logical reason to do so. In the "wild west" of research software, where commit messages are often uninformative, it is not possible to rely on a human to label a change as a "bug fix."

A substitute that this paper explores is replacing missing semantic data with a heuristic based approach where file risk gets identified through metrics. This necessitates shifting the focus from why a file changed to how much it changed and what it changed with. Setting the focus for the heuristics to be temporal while ignoring the structural. Using the scale of the data to supplement the natural loss of fidelity that this created. Naturally leading to the study of Code Churn.

3.2.3 Code churn's impact on maintainability

Code churn is the rate of change of lines of code in a file. A file with high churn might have an incredibly high number of additions and deletions while remaining small in terms of LOC. This is an ATD hotspot when assessing points of interest in terms of maintainability in codebases. (Faragó, Hegedűs and Ferenc, 2015) is paper which assess how code code churn affects the maintainability of codebases, by assessing the code churn of files in large codebases, achieving this by calculating the sum total of lines added and deleted on every file. Supplementing this the maintainability of the code was measured using ColumbusQM probabilistic software quality model (Bakota *et al.*, 2011). The paper was able to assess the levels of maintainability per file in proportion to the amount of code changes, drawing the conclusion that code that has high churn is code in which it is harder to maintain. This conclusion makes sense, it reaffirms that poorly architected code will need more changes which increases the maintainability burden of the code. The method in which the code churn was measured is quite useful and will be the approach taken for this study.

Unfortunately there is one key issue with this paper, the use of ColumbusQM for assessing the maintainability of the codebase. ColumbusQM is a statistical machine which takes metrics such as lines of code, styling, coupling, number of incoming invocations etc.. Aggregates them which then computes a numerical output. The metrics such as styling are something so subjective and minor that the inclusion as one of the key metrics in the evaluation undermines the whole statistical model. Taking into account all of the evaluation metrics this is a case of blind assessment where structural relations or evolutionary dependencies are not measured and only a snapshot of the codebase is measured.

Taking these points into account, the key takeaway from (Faragó, Hegedűs and Ferenc, 2015) is the methodology regarding code churn, rather than its conclusions on maintainability. By identifying the intersection of Modularity Violations (Structural) and High Churn (Temporal), a composite metric can be created that avoids the "blindness" of ColumbusQM. This allows us to quantify architectural decay using only version control

metadata, remaining entirely agnostic to both the quality of documentation and the subjective "style" of the researcher. Giving us the foundation of files to be used to create opinionated static analysis tooling.

3.3 Static Analysis

The transition from historical temporal analysis to active development requires a mechanism that can evaluate code in its current state, rather than its past. Static Analysis serves as this mechanism, offering a storied field of research focused on improving code quality by examining source code without execution. While the temporal analysis identifies the "symptoms" of architectural decay through commit history, static analysis allows us to identify the specific structural patterns that lead to high maintenance overhead.

3.3.1 Current Methods

Modern methods for static analysis include data flow analysis, syntactic pattern matching, and abstract interpretation (Gosain and Sharma, 2015). These methods have evolved from simple token-level scanning to sophisticated project-level reasoning, such as path analysis and control-flow graph (CFG) mapping. Current state of the art tooling such as Codescene has made strides in moving towards behavioural code analysis. Codescene's creation of "refactoring targets", a system in which traditional static analysis is used to identify high risk files is overlaid with a heatmap of developer activity to focus in on problematic active files. Addressing a fundamental issue of traditional static analysis: the "urgency" gap. This approach is grounded in the principle that tech debt is only an immediate risk if it exists in "active code". These "refactoring targets" provide a pragmatic prioritisation of maintenance effort allowing developers to focus on the 1-3% of the codebase that typically accounts for the majority of maintenance overhead.

3.3.2 Issues with Modern Approaches

Despite this increase in analytic capabilities, noise pollution in which there are too many warning messages discouraging any actions being taken to remedy them. The same issue which plagued Lint (Johnson and Hill, 1978) is still an issue to this day. Polluting the developer experience creating friction and actively discouraging developers form using the tools as it becomes more effort to sift through and find the real bugs than fixing them normally (Johnson *et al.*, 2013). Leading to the conclusion that the research community, especially due to the lack of experience in the field (Hannay *et al.*, 2009) would have more difficulties differentiating fact from fiction and discouraging them from using static analysis at all. This concretely highlights the difficulties of modern static analysis tooling in a research context. The focus on a general one size fits all tool means that it lacks the ability to be opinionated by default. The gap this presents is one where research software would have a bespoke set of curated rules reducing noise and guaranteeing improvements in code quality once executed.

This study poses that the key issue with the analysers in the case of false positives is not the rules but how they were derived. Historically most academic research in

software engineering has required significant empirical evidence to be considered by industry (Weyuker, 2011), yet the current landscape is dominated by expert analysed rules. The challenge of software maintainability with static analysis is elusive. "Ghost Echoes Revealed" highlights this (Borg, Ezzouhri and Tornhill, 2024), critiquing the "ghost echoes" produced by traditional expert-derived static analysis. These echoes are not objective truths, but rather technical artifacts, static rules that linger in codebases despite often failing to align with the lived experience and thoughts of human developers.

By benchmarking machine learning models against human judgment, the study reveals a fundamental difference: expert rules frequently draw "hard lines" that humans do not actually perceive as problematic. This suggests that codebase governance based on these traditional metrics is less an exercise in objective optimization and more an adherence to institutional haunting. The precedent set here is that for codebases to remain truly maintainable, developers must move beyond the rigid, subjective opinions of experts and toward empirically validated models that reflect the nuanced, non-deterministic reality of human-centric code health."(Borg, Ezzouhri and Tornhill, 2024)

3.3.3 Rules as Data Approaches

The limitations of expert-derived thresholds are clear and reveal the need for a paradigm shift from deductive to inductive static analysis. Traditional tooling is deductive, applying a pre-defined expert rule as a universal threshold to a specific file to deduce a "bug" or "code smell". In contrast, a rules as data approach treats the repository itself as the source of truth with its history as well as the current state being imperative to identifying architectural technical debt. (Borg, Ezzouhri and Tornhill, 2024)

Machine learning has the capabilities to identify patterns in code, how it evolves, how it decays, with this holistic view an inductive approach can be taken. In which the "rules" are taken not from expert opinion but from data taken from the wider domain. Creating curated opinionated rules tailored to a specific domain, minimising the "ghost echoes" of irrelevant "universal" rules created for industry, replacing them with empirically validated opinionated guardrails on code quality.

3.4 Machine Learning

Machine learning is a pivotal technology within the context of this paper. Allowing rigorous statistical analysis to be carried out to create objective metrics for the maintainability rules. Machine learning provides the framework for rigorous statistical analysis, allowing for the creation of objective metrics. These algorithms operate by minimizing an objective function(A mathematical representation of error). An example would be decision trees, this is achieved through greedy splitting, where the algorithm recursively partitions data to minimize "impurity" (uncertainty) at each step. By optimizing for the lowest possible error, the model becomes increasingly proficient at identifying patterns within the problem domain. There are many approaches and models available but due to an emphasis put on human readability, decision trees and random forest best fit this use case.

3.4.1 Decision Trees

Decision trees are a standout choice when it comes to interpretable machine learning models. They consistently rank highly within classifier models while still supporting rule extraction which aids readability. Decision trees represent classifiers as hierarchical IF-THEN rules, where each root-to-leaf path corresponds to a conjunction of feature thresholds that directly maps to executable static analysis checks. A concrete example of this would be IF nesting > 3 THEN unmaintainable; in the case of nesting being over 3 then the code would be labeled unmaintainable. This simplistic example would be scaled up much larger with many more decision and leaf nodes creating a classifier based on simple decisions at each step. Naturally there are some drawbacks to this approach, most prominently the overfitting problem. An overfitted model reflects the structure of the training data set too closely. Even though a model appears to be accurate on training data, it may be much less accurate when applied to a current data set (Khoshgoftaar and Allen, 2001). This issue restricts their usage in current maintainability prediction methods, (Bluemke and Borsukiewicz, 2023) shows that Decision Trees can prove useful in a baseline analysis of maintainability predictors. Despite limitations, DT rules provide the simple interpretability missing from random forests, enabling manual validation of learned static checks against domain knowledge. Pruning or ensemble averaging in random forests addresses overfitting to a degree while retaining path-derived rules for analysis (Quinlan, 1993).

3.4.2 Random Forest

Random forest is an approach derived from decision trees and a popular classifier machine learning approach. Relying on creating a set of decision trees (called a forest) each of which vote on the outcome to choose a class for the classifier, it creates a method that leverages the law of large numbers to deal with the overfitting problem in decision trees (Breiman, 2001) in addition to increasing accuracy. The method in which overfitting can be largely ignored is that there are many trees voting, every one could potentially be overfitted yet the whole forest remains generalised to the problem as every tree would be overfitted to a different feature. Random forests average better results theoretically (Breiman, 2001) and practically in the field of maintainability research (Bluemke and Borsukiewicz, 2023) over decision trees. The key drawbacks to random forests are both the increased computational costs and the black box view when it comes to readability. Modern research (Haddouchi and Berrado, 2025) has made progress in creating methods of rule extraction for random forests as before the size of the forests rendered them as a black box approach. (Haddouchi and Berrado, 2025) focused on deriving a rule ensemble based on the calculated weighted importance of the trees. Allowing for a dimensional reduction method to be applied to the forest. Within the study an average factor of 300x reduction to the number of rules per target class both in binary and multi classification problems was observed while retaining a 93% accuracy compared to the full forest's 95% accuracy. Allowing a highly optimised set of rules to be generated for both performance and explain ability. Leveraging this rule extraction method for random forests is pivotal to create a rule set that properly generalises

across projects while retaining the core idea and performance of the model and keeping with the core ideal of readability.

3.4.3 Applications to maintainability detection

The application of machine learning to maintainability detection enables a data-driven approach to identifying code structures that hinder long-term quality and evolution. Traditional static analysis methods rely on human-curated heuristics, which often fail to generalise across diverse projects or languages. In contrast, machine learning can infer maintainability rules directly from empirical evidence, allowing patterns of poor or high maintainability to emerge statistically rather than being predefined. State of the Art(Sota) models achieve impressive results in classification of low maintainability files. (Bertrand *et al.*, 2023) achieved an 82% F1 score with an AdaBoost classifier, while (Bluemke and Borsukiewicz, 2023) achieved 93% F1 score with a random forest classifier and (Schnappinger *et al.*, 2019) achieved an F1 score of 80% using a J48 classifier across three labels. However, each approach shares a critical limitation, they rely on human experts to create a ground truth of annotated data. PROMISE (Boetticher, Menzies and Ostrand, 2007), MainData (Schnappinger, Fietzke and Pretschner, 2020) and a custom expert labelled dataset (Schnappinger *et al.*, 2019) were used in these studies. This introduces a certain degree of subjectivity which can be minimised by increasing the number of reviewers although limiting scale. These datasets are also by design due to the time commitment labelling, older and cannot reflect the current programming landscape. This limitation is the key motivation for an algorithmic approach to labelling, allowing us to create a dataset based on current codebases at scale.

A crucial step in applying these machine learning models to maintainability detection is the transformation of source code into a machine-readable representation. Abstract Syntax Trees (ASTs) provide this structure, encoding the syntactic and hierarchical relationships between program elements. By traversing or analysing the AST, a wide range of numerical and categorical features can be extracted: such as nesting depth, branching density, or average method size, which serve as the input features for machine learning models (Bertrand, Favier and Andre, 2022). Various learning models can then be trained on these AST-derived metrics to classify code fragments according to maintainability characteristics.

3.5 Summary

The machine learning approach offers a compelling alternative to expert-derived static analysis rules. By using interpretable models such as Decision Trees, we can extract human-readable rules that map directly to executable code quality checks. Random Forests provide higher accuracy but require additional effort to extract interpretable rules. Prior work has demonstrated strong classification performance (82-93% F1), but these approaches rely on expert-labeled datasets that are difficult to scale and may reflect outdated coding standards or domain specific labelling.

This study takes a different approach: rather than relying on human experts to label data, we use the Hub Score, an objective metric derived from version control metadata, to automatically label files as "High Risk" or "Low Risk." This automated labeling enables us to train machine learning models on a larger, more diverse dataset without the overhead of manual expert annotation. The resulting model learns which structural characteristics (nesting depth, complexity, coupling) correlate with files that developers actually change frequently, translating these patterns into static analysis rules that are empirically grounded rather than opinion-based.

4 Methodology

This study follows an empirical, data-driven pipeline to derive static analysis rules from real-world research software. The process is divided into four primary phases. First, we aggregate a dataset of C++ repositories from the Journal of Open Source Software (JOSS). Second, we employ a custom-built tool to extract evolutionary coupling metrics and calculate a Hub Score for each file, providing an objective "High Risk" or "Low Risk" label. Third, these labeled files are used to train a Decision Tree model to identify the structural characteristics of high-risk code. Finally, the logical paths within the trained model are translated into human-readable static analysis rules, bridging the gap between historical developer behavior and proactive code quality standards.

4.1 Dataset Selection and Acquisition

The primary goal of this paper is to derive objective rules to raise the level of coding standards in the research software scene. This lends itself to using research software as the dataset. The Journal of Open Source Software (JOSS) (*Journal of Open Source Software*, no date) was selected as the primary source due to its peer-review requirement, which ensures a baseline of documentation and code quality.

4.1.1 Data collection

A custom utility tool in rust was created to aggregate metadata from JOSS. This tool interfaced with Github API to gather repository-level metrics(contributor count, collaborator count). The resulting data was analysed by a python pipeline with pandas (team, 2020) to determine the final dataset. To ensure sufficient architectural complexity, projects were filtered based on the following criteria:

- Collaborator Threshold($>= 4$): Projects with fewer than four collaborators often reflect individual coding styles rather than standardized maintainability practices. A minimum of four contributors ensures a level of communication overhead that requires formal architectural patterns. As seen in Figure 1 the majority of repositories remain after this filtering, while low-developer outliers are removed.

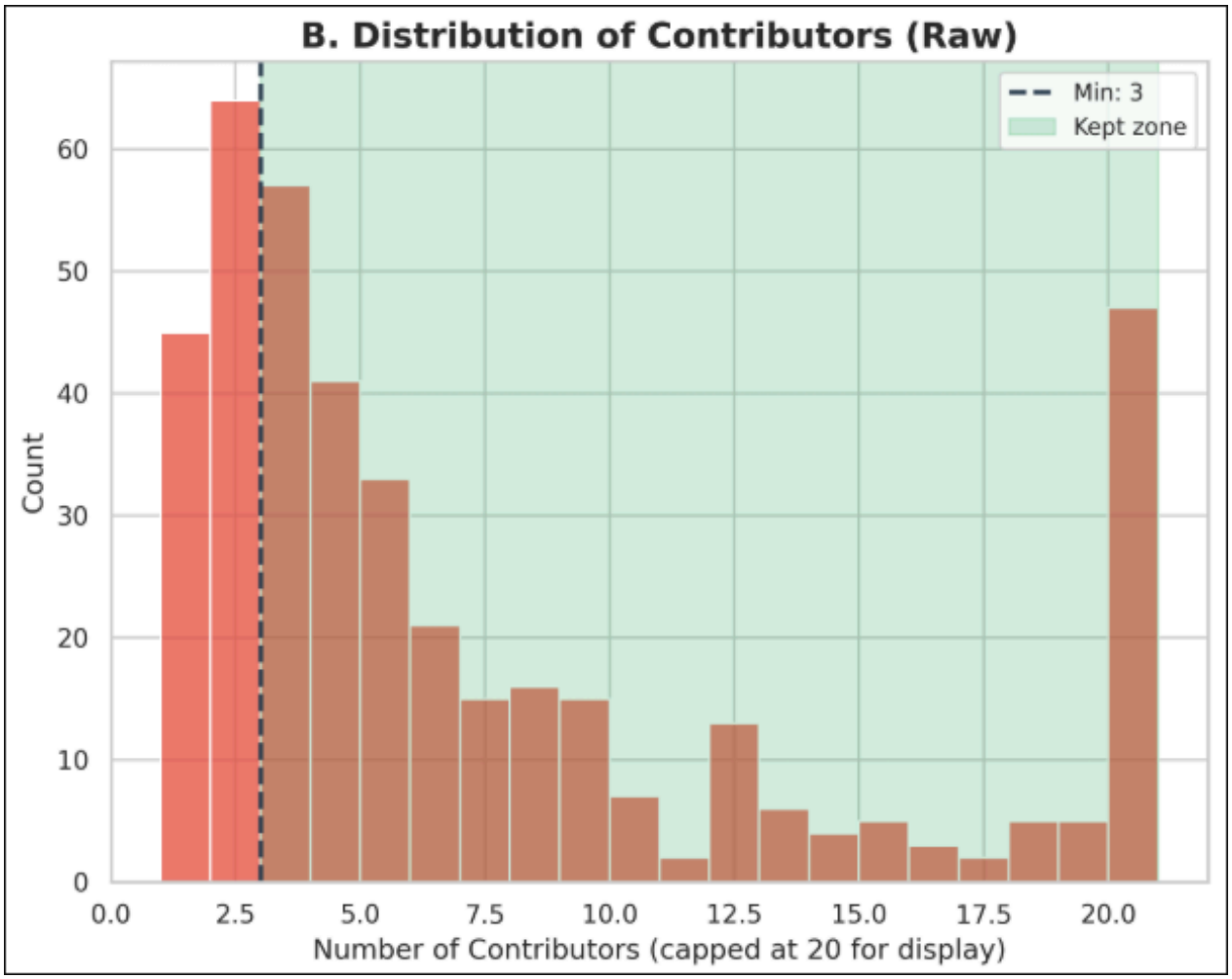

Figure 1: Pre filter Distribution of Collaborators

- Commit Volume (100–5,000): To ensure the dataset captures meaningful maintenance behavior, a "maturity floor" of 100 commits was established. Conversely, a "computational ceiling" of 5,000 commits was enforced to maintain feasibility on local hardware. As seen in Figure 2, these thresholds effectively remove the "short-tail" of embryonic projects and the "long-tail" of infrastructure-scale outliers. This dual-sided filtering retains the vast majority of the JOSS population while ensuring a consistent and processable data scale.

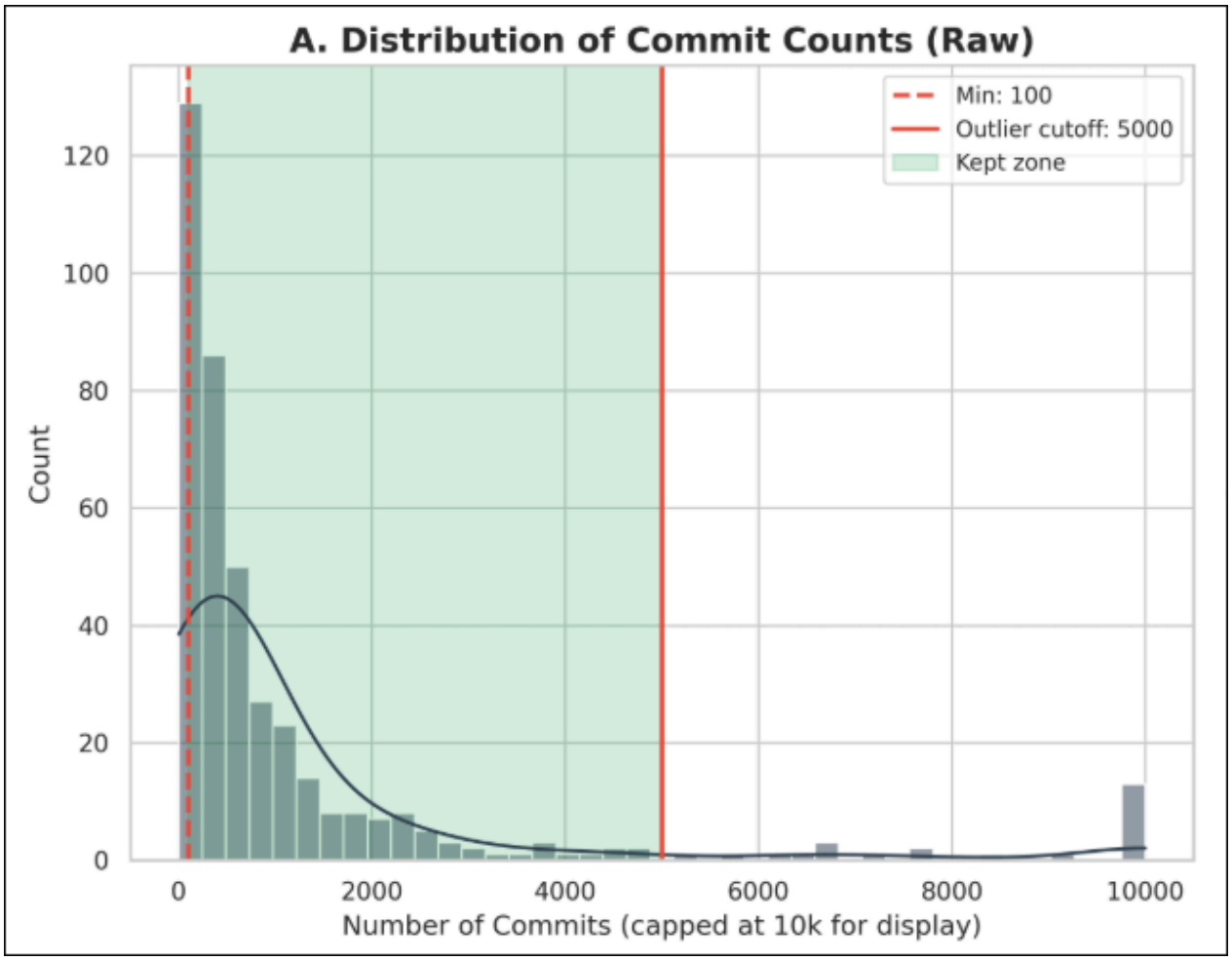

Figure 2: Pre filter Distribution of Commits

- Target Language: C++ was selected as the target language due to its dominance in high-performance research computing and the specific maintainability risks associated with its manual memory management and low-level abstractions. Restricting the study to a single language ensures methodological consistency, as code metrics are often non-comparable across different programming paradigms and syntax structures.

This filtering took the original 406 repos down to 213 which is a significant reduction in dataset size, those projects primarily consisted of individual small scale work that lacks the depth and collaboration overhead required to test maintainability work.

4.1.2 Dataset First Pass Analysis

To categorize the dataset, automated labeling heuristics were utilised rather than traditional expert manual review. While expert opinion is often the standard for establishing "ground truth", it is difficult to scale across thousands of research repositories. Instead, the tool analyzed the historical behavior of each file to assign a Hub Score: a composite metric that measures a file's "dependency" within the project. This score is calculated based on three key factors: the number of coupled files, the average coupling

ratio, and the file's overall code churn. These metrics were gathered by running a custom analysis tool over the entire commit history of each repository. The tool constructs a Co-change Graph, where Nodes represent individual files and Edges represent shared commits between files. The graph allows us to evaluate the coupling of each file by comparing the number of times it was modified alongside other files versus the number of times it was modified in isolation.

To map the evolving relationships between source files a custom rust tool was created. Due to the scale of the relationships between files that needed to be tracked an in memory solution was not feasible, therefore Neo4j (Technology, 2015), was an ideal solution. As a graph database, Neo4j is uniquely suited for managing the complex, non-linear relationships inherent in software evolution. The tool performs a sequential traversal of the repository's entire commit history from the initial root, to the current head. At each commit tracking the codependency between each file as they were committed over time. This created a robust database of the temporal dependencies of the repository's architectures. A key technical challenge arose over large commits. In research software researchers wouldn"t be aware of commit hygiene (Hannay *et al.*, 2009) leading to large scale "bulk" commits. If a dependency such as a dataset were uploaded of 1,000 files, that could generate 1,000,000 relationships to track. To maintain performance and ensure data relevance, a threshold was established to skip any commit containing more than 100 files. This optimization focuses the database on intentional, developer-driven architectural changes rather than bulk file operations.

The output of the analysis pipeline was a Neo4j graph database, which served as the computational foundation for calculating file-level maintainability risks. By leveraging graph-traversal queries, the dataset was categorized using a composite heuristic "Hub Score."

4.2 Dataset Labeling

Drawing on three complementary insights from prior work, this paper proposes a composite Hub Score that synthesizes temporal coupling and change frequency into a single normalized metric. (Xiao *et al.*, 2016) demonstrated that commit coefficients between files serve as reliable indicators of architectural debt. Files that frequently change together despite lacking structural dependencies represent "Modularity Violations" that accumulate ATD. Building on this, (Faragó, Hegedűs and Ferenc, 2015) showed that files exhibiting high code churn (frequent line additions and deletions across commits) correlated directly with reduced maintainability. Finally, (Cai *et al.*, 2025) empirically validated that coupling density measured by the number of partner files directly increases maintenance overhead across 1,200 Google repositories.

As noted by (Xiao *et al.*, 2016), focusing primarily on modularity violations ignores other potential risk factors. However, this approach is justified by the unique scale of our study. Unlike traditional, manually-curated datasets such as PROMISE (Boetticher, Menzies and Ostrand, 2007) or MainData (Schnappinger, Fietzke and Pretschner, 2020),

our automated "broad-spectrum" filtering allows us to identify the most critical offenders across a significantly larger volume of data. By combining these repositories, a dataset was created that is both objective and massive in scope, providing a high-quality surface area for supervised learning using "High Risk" and "Low Risk" labels.

4.3 Feature Selection.

To quickly calculate a relative score of each file this formula was used to calculate a value called a Hub Score which represents the temporal coupling and change rate of every file in compared to others.

4.3.1 Hub Score

$$\text{HubScore} = |(C) \times \left(\frac{P}{F_t}\right) \times \left(\frac{W_f}{W_t}\right) \tag{2}$$

Variable Definitions

- $|(C)$ (**avg_coupling**): The mean strength or frequency of dependencies between this file and its partners.
- P (**partner_count**): The number of unique files that this specific file is coupled with.
- F_t (**total_files**): The total number of files in the repository or local subset.
- W_f (**file_churn**): The number of changes or commits associated with this specific file.
- W_t (**total_churn**): The total sum of changes across the entire repository.

This is a bespoke formula normalising per file metrics proportionally to repository wide metrics to allow accurate comparison across repositories.

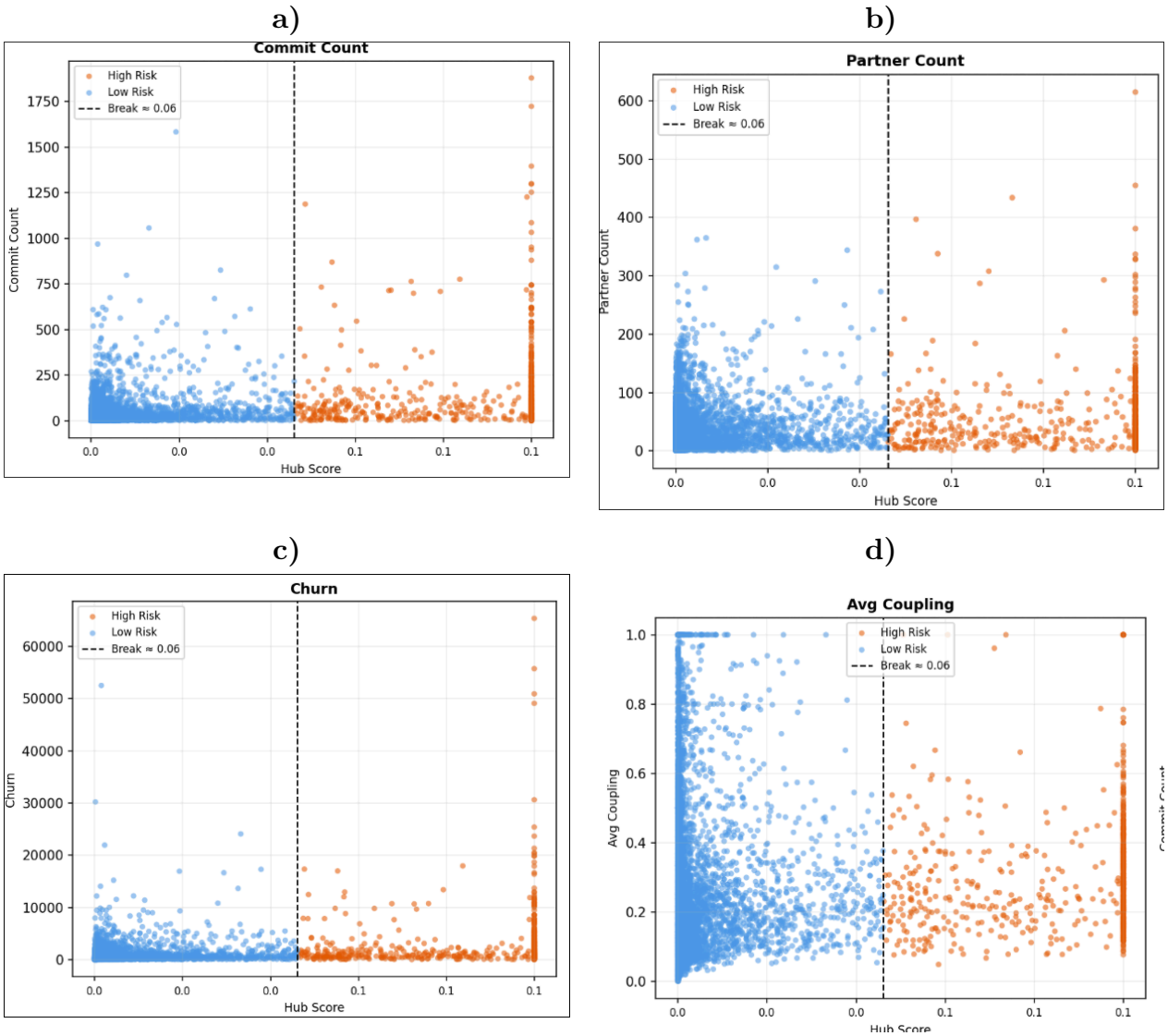

Figure 3: Hub Score Distribution across metrics: **a)** Commit Count, **b)** Partner Count, **c)** Code Churn, and **d)** Average Coupling.

The relationship between repository metrics and the Hub Score is visualized in Figure 3. While a positive correlation exists across all variables, the dispersion across each metric confirms that the formula successfully normalizes for scale, preventing larger repositories with more history from automatically receiving high scores. This ensures that the identification of technical debt is context-aware and comparable across repositories of varying scales

The core justification to use this formula is derived from three core principles:

1. A logic file should not have to change often. Supported by the Open Closed principle (Meyer, 2009) and code churn analysis (Faragó, Hegedűs and Ferenc, 2015), which demonstrated that high frequency and large size of changes correlate to an increased maintenance overhead.

2. A logic file should not be highly coupled to many partner files. (Cai *et al.*, 2025) empirically validated that coupling density directly increases maintenance overhead across 1,200 Google repositories.
3. Files that change together despite lacking structural dependencies represent hidden architectural debt. (Xiao *et al.*, 2016) identified these "Modularity Violations" as the most common and expensive form of architectural debt, where files co-change without explicit dependencies.

This three-principle justification is not merely theoretical; it mirrors the approach of prior work that uses evolution history as ground truth for supervised learning. Demonstrated that co-change information can produce higher-quality labels for code smell detection than expert annotation alone, validating the use of temporal coupling as an objective labeling mechanism for machine learning.

4.3.2 File classification and oversampling

To prepare the data for a binary Decision Tree classifier, the continuous Hub Score was split into two categorical classes: High Risk and Low Risk. Rather than applying an arbitrary numerical threshold, Jenks Natural Breaks algorithm was employed to identify the ideal split point (Jenks, 1967). This algorithm identifies natural groupings within the data by minimizing the variance within each class while maximizing the statistical separation between the classes. Ensuring that high risk labels are assigned to architectural outliers rather than files marginally exceeding a mean or median value.

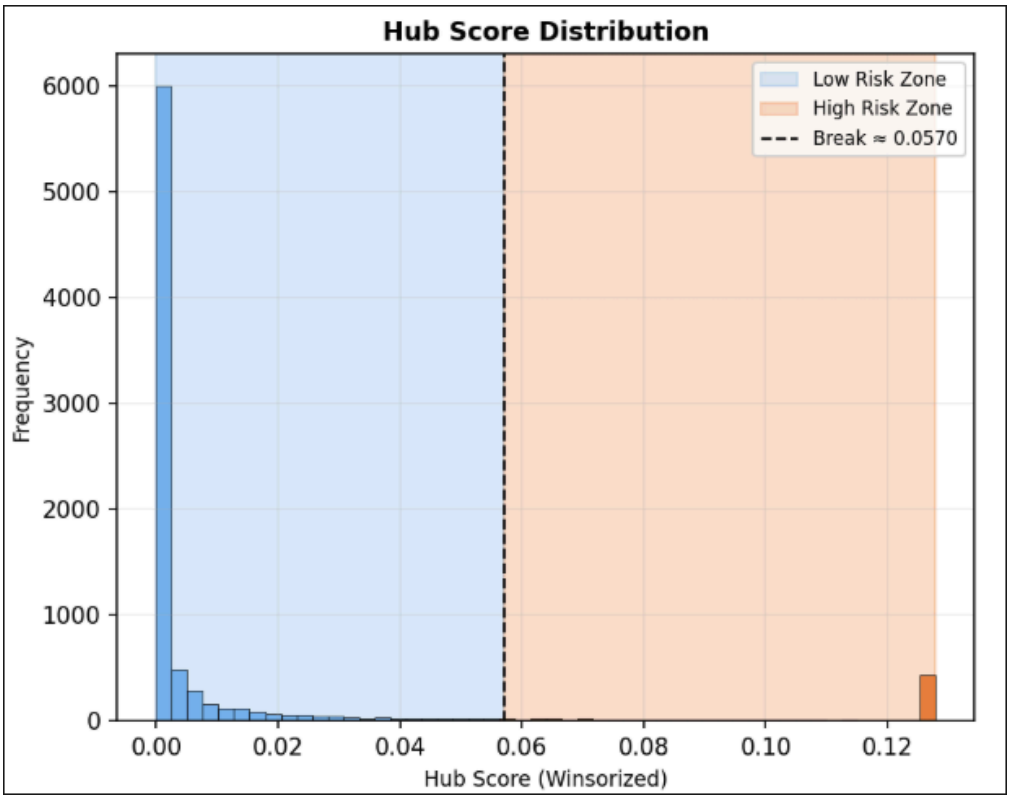

Figure 4: Hub score distribution across all .cpp samples

As normally observed in software the distribution of architectural debt is often skewed, with a small number of files contributing to the majority of risk and maintenance overhead, the same separation was observed within this dataset with 5100 files classified as low risk and 637 files classified as high risk. Figure 4 With a disparity this large between labels and the need for a dataset when performing binary classification to avoid a majority class bias towards low risk files, Synthetic Minority Over-sampling Technique (SMOTE) was implemented. Following the precedent for using SMOTE set by (Bluemke and Borsukiewicz, 2023) in maintainability prediction, SMOTE generates synthetic observations within the feature space rather than simply duplicating existing records (Chawla *et al.*, 2002). Providing the model with a balanced training environment and significantly improving the recall of high-risk files without compromising the classifier's ability to identify maintainable code.

4.3.3 Dataset verification method

To address the validity of the autonomous labeling process (RQ1), a verification step was performed to reveal the extent Hub Score aligns with established architectural risk benchmarks.

To test this Codescene was used as a ground truth for architectural technical debt. odeScene was utilized as the ground truth for architectural technical debt, which categorizes file health into three tiers: Healthy (Health > 9), Problematic (Health 4–9), and Unhealthy

(Health < 4). Following the definition established by (Borg, Ezzouhri and Tornhill, 2024), both 'Problematic' and 'Unhealthy' classifications were aggregated into a single 'High Risk' label for the purpose of this study. To compliment this, a second comparison will be made with only 'Unhealthy' files to see if the hub score is similar to a 'high pass filter' effectively isolating critical architectural risks while ignoring moderate structural issues that may not yet manifest as evolutionary bottlenecks.

The verification process involved a set of repositories from (*CERN/awesome-cern*, 2026) which were used to calculate industry standard labels for high risk for architectural debt per file. A custom extraction tool was created to retrieve architectural data from the codescene.io which was then subsequently put into standardised CSV structure for comparison.

To generate the corresponding Hub Score labels, each repository was assessed individually. A static threshold was derived for each repository using the Jenks Natural Breaks algorithm. This allowed for the autonomous separation of files into binary classes, High and Low Risk, based on their specific Hub Score. These were then cross-referenced against the CodeScene-derived labels for a final comparison of the extent of similarity between both datasets showing the efficacy of the hub score method.

4.3.4 Addressing Data Leakage

A critical consideration in this methodology is the separation of Labeling Criteria and Training Features. The Labels are derived from Temporal Metadata (commit history, co-change frequencies, and historical churn). The Features (inputs for the Decision Tree) are derived strictly from Static Source Code Analysis (e.g., McCabe Complexity, LoC, Nesting Depth). This separation ensures that the model is learning to predict future maintenance risk based on the current state of the code, rather than simply "re-discovering" the components of the Hub Score formula.

4.3.5 Dataset Summary

The final dataset consists of 9,000 lablled files from 200+ research repositories distributed across each repository The dataset was generated entirely automatically using the custom rust and python tooling using the empirically defined method. There is a strong class imbalance between high and low risk with a majority bias towards high risk requiring the models to learn meaningful relationships to label a file as high risk.

4.4 Feature Engineering

To prepare the high risk and low risk datasets for machine learning, the raw source code files had to be converted into tabular data compatible with decision trees and random forests. Feature extraction was performed using the rust-code-analysis (RCA) crate (Ardito *et al.*, 2020). RCA is used on the base source files to statically derive numerical metrics on the files, the metrics can be seen in Table 1

Metric	Description
CC	McCabe's cyclomatic complexity: it calculates the code complexity examining the control flow of a program. It is measured as the number of linearly independent paths through a piece of code.
SLOC	Source lines of code: it returns the total number of lines in a source file.
PLOC	Physical lines of code: it returns the total number of instruction and comment lines in a source file.
LLOC	Logical lines of code: it returns the total number of logical lines (statements) in a source file.
CLOC	Comment lines of code: it returns the total number of comment lines in a file.
BLANK	Blank lines: it counts the number of blank lines in a source file.
Halstead	The Halstead suite, a set of seven statically computed metrics, all based on the number of distinct operators (n1) and operands (n2) and the total number of operators (N1) and operands (N2). The suite provides a series of information, such as the effort required to maintain the analyzed code, the size in bits to store the program, the difficulty to understand the code, an estimate of the number of bugs present in the codebase, and an estimate of the time needed to implement the software.
MI	Maintainability index: a suite to measure software's maintainability, calculated both on source files and functions.
NOM	Number of Methods: it returns the number of methods in a source file.
NARGS	Number of arguments: it counts the number of arguments of each method in a source file.
NEXITS	Number of exits: it counts the number of possible exit points of each method in a source file.

Table 1: Extracted code metrics (Ardito *et al.*, 2020)

Prior to model training the extracted features underwent additional preprocessing to enhance performance. Columns containing NaN values were removed, as certain language-specific metrics were non-applicable to the C++ source files within the dataset. The Maintainability Index (MI) column was dropped to remove potential bias from previous maintainability prediction methods. Low variance (<0.01) and highly correlated features ($|r| < 0.9$) were dropped to reduce the search space when selecting variables. Simple ratios between relevant features (Coleman *et al.*, 1994) were computed such as operator operand ratio to capture relative relationships between measurements. Lastly due to the nature of tree based machine learning models further metrics were calculated using sklearn's (Pedregosa *et al.*, 2011), polynomial feature generation on key numerical metrics to allow non linear metrics to be an option for the tree model feature selection.

4.5 Model Implementation and Training

Following the refinement of the feature set, the processed tabular data was utilized to train and validate two supervised learning models: a Decision Tree and a Random Forest. These models were selected for their inherent interpretability, allowing the extracted maintainability metrics to be mapped to clear, logical decision boundaries.

To ensure a robust evaluation of both models, a standardised training approach was used. Sklearns's (Pedregosa *et al.*, 2011), DecisionTreeClassifier and RandomForestClassifier were used as the base models. GridSearchCV was used to perform hyperparameter fine tuning on the models where F1 score was used as the primary optimisation target ensuring a balanced performance metric between high and low risk files. A 5-fold stratified cross-validation was used over a traditional train-test split to ensure that each fold remained representative of the overall class distribution, mitigating the risk of sampling bias that can occur with a single split Coupled with this to aid in the recall of the model to lower the chance of a false negative, a weighted cost function was applied, penalizing the misclassification of high-risk files more heavily than low-risk files.

The optimal configurations for decision trees and random forests are detailed in the results section. Table 3

4.6 Rule Derivation Process

The selection of tree-based models was driven by the requirement for rule extraction. Once the models reached optimal performance, two distinct methodologies were used to translate the mathematical weights into human-readable coding standards:

- Decision trees: Decision trees provide an inherent "IF-THEN" logical structure. To ensure interpretability, the models were constrained by maximum depth parameters during training. The trained model was exported as a visual representation using scikit learns's tree visualization tools (Pedregosa *et al.*, 2011), allowing for a clear view of the decision paths.
- Random Forests: Due to the complex ensemble nature of random forests only a derivation of the rules can be extracted (Haddouchi and Berrado, 2025). Using the tool te2rules (Lal, Chen and Mithal, 2024), an approximation of the random forests rules can be generated to a high fidelity providing a human-readable summary of the ensemble's collective logic.

The extraction of human readable rules is a strong step towards practical application. However they cannot be trusted without empirical testing as without it they are worse than expert rules.

4.7 Evaluation Framework

To determine the validity of the ML-derived rules testing against the current Sota in static analysis is required, for which Codescene was the tool selected. Using three case studies

selected from the Awesome CERN (*CERN/awesome-cern*, 2026) page as repositories fitting the criteria

To evaluate the performance and generalisability of the proposed models, a multi-stage validation framework was implemented. Initial model robustness was assessed using 5-fold cross-validation, with the F1-score serving as the primary metric to ensure a balanced evaluation of precision and recall.

To test the models“ utility in real-world scenarios, an external validation was conducted using a series of “case study” repositories held entirely separate from the training set. The case studies were selected from the CERN Awesome page (*CERN/awesome-cern*, 2026) as CERN produces large scale open source software hitting the same criteria as the repositories in the training set. For these repositories, industry-standard analysis from CodeScene was used as the ground truth. The random forest model was run against these ground truth labels to derive an F1 score to assess the application of the models in the wider research software development scene.

4.8 Summary

The methodology presented here establishes a complete pipeline for empirically deriving maintainable, opinionated static analysis rules from software evolution data. Leveraging the scale of 212 different research repositories to generate a dataset using objective metrics by combining temporal coupling analysis with structural metrics. Using machine learning approaches to produce explainable rules that reflect actual developer behavior rather than expert opinion.

5 Results

NOTE: I will change wording when i get more case studies done, ran into some issues but want feedback on structure of section as if I had multiple results

This section will present the performance of the data labelling performed using the Hub Score metric, the F1 score of the models trained on the dataset, the efficacy of the extracted rules and the comparison to current static analysis tooling.

5.1 Hub Score Comparison with Codescene

Looking at RQ1, to accurately assess the extent that version control metadata can autonomously label architectural risk in domain specific repositories a case study was performed. The code health metrics from Codescene when using the code health < 9.0 set out from (Borg, Ezzouhri and Tornhill, 2024) when compared to the hub score metrics led to an insightful conclusion. Table 2 reveals that the inclusion of problematic files ($4 <$ health < 9) is something that hub score cannot detect but the truly high risk files (health < 4) codescene and hub score both returned 0. Revealing that the hub score composite metric can emulate the structural identification criterion using only temporal metrics as a high pass filter for only very high risk files.

	ACTS
Total Files	876
HR by Hub Score	0
HR by Code Health < 9	147
HR by Code Health < 4	0

Table 2: High risk identification comparison Codescene vs Hub Score

5.2 Machine Learning Models

5.2.1 Model Configuration

To ensure the results were not the product of sub optimal configuration, GridSearchCV was employed to identify the optimal parameters using F1 score as the objective function. This was conducted on both the decision tree and random forest models and the final configuration can be seen here Table 3

Parameter	**DT Value**	**RF Value**
Class Weight	False: 1, True: 2	False: 1, True: 3
Criterion	entropy	N/A
Max Depth	7	30
Max Features	N/A	sqrt
Min Samples Leaf	5	4
Min Samples Split	10	10
N Estimators	N/A	300

Table 3: Model hyperparameters

5.2.2 Model Performance

Using the optimal configuration in Table 3 5 fold stratified cross validation on the SMOTE-balanced training dataset was conducted. Decision trees achieved an F1 score of (34%) and random forest achieved an F1 of (42%).

Model	Precision	Recall	Accuracy	F1
Raw Dataset				
Decision Tree	—	—	—	—
Random Forest	—	—	—	—
SMOTE Dataset				
Decision Tree	27%	46%	76%	34%
Random Forest	34%	54%	80%	42%

Table 4: Model Performance Comparison

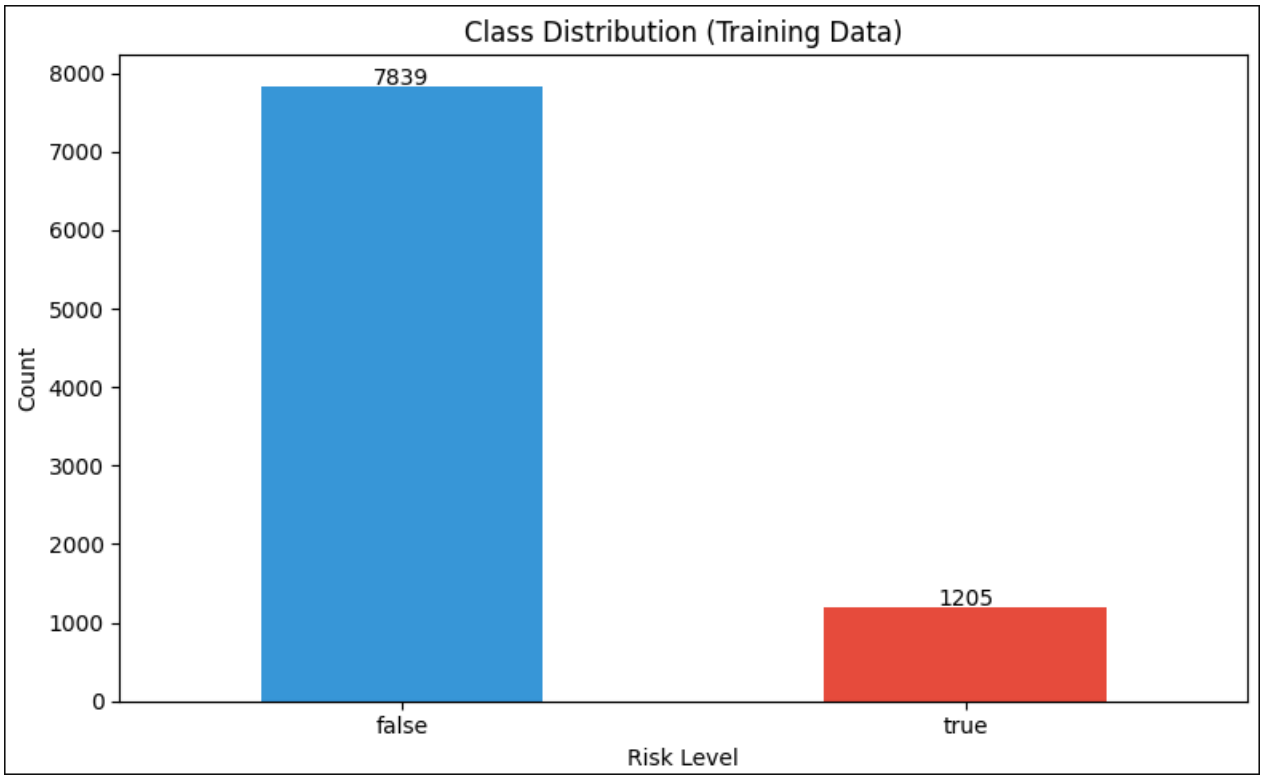

Figure 5: Distribution of high and low risk files

Looking at Table 4 across both models there is a here is a significant disparity between the high Accuracy (76%) and the low F1 scores, particularly in the Decision Tree (34%). This aligns with the dataset imbalance noted in Figure 4 and Figure 5 where while training SMOTE provides a richer training set the test set is representative of the real data which favours a bias towards predicting low risk for the highest accuracy. Meaning accuracy is simple to raise by predicting only low risk but the truly valuable models will be able to accurately predict the outliers to the class imbalance.

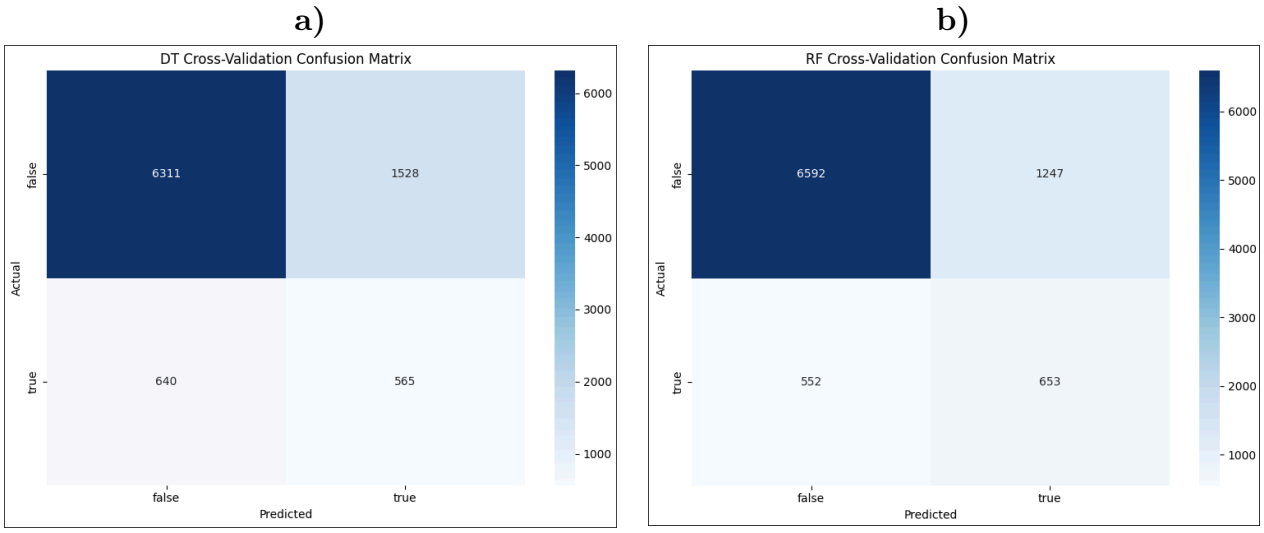

Figure 6: Confusion Matrices of predicted metrics: **a)** Decision Tree SMOTE, **b)** Random Forest SMOTE,

Figure 6 reinforces the divergent strategies of the two models. The Decision Tree demonstrates a 'Hyper aggressive' strategy maintaining accuracy and recall by sacrificing precision by predicting high risk a disproportionately high amount of the time, higher even than the total number of high risk samples in the dataset were incorrectly predicted. Conversely, Random Forests demonstrates a more 'Conservative' strategy leveraging the additional predictive power of the ensemble of trees it achieved a much higher recall by more accurately being able to predict high risk files more readily achieving a recall of (54%) in comparison to the Decision Tree's (46%). As evidenced by Figure 6, both models produced a higher volume of False Positives than True Positives. For the Random Forest, the precision of 34% indicates that for every 10 files flagged as 'High Risk,' approximately 6 to 7 are likely stable files. This 'Ghost Echo' effect remains a significant barrier to developer adoption, as the tool effectively generates more 'noise' than actionable 'signals' (Johnson *et al.*, 2013)."

5.2.3 Feature Importance

To understand the underlying logic of the classifiers, the Feature Importance was extracted from the Decision Tree and Random Forest model. This identifies which static metrics provided the strongest signal when predicting the temporal Hub Score.

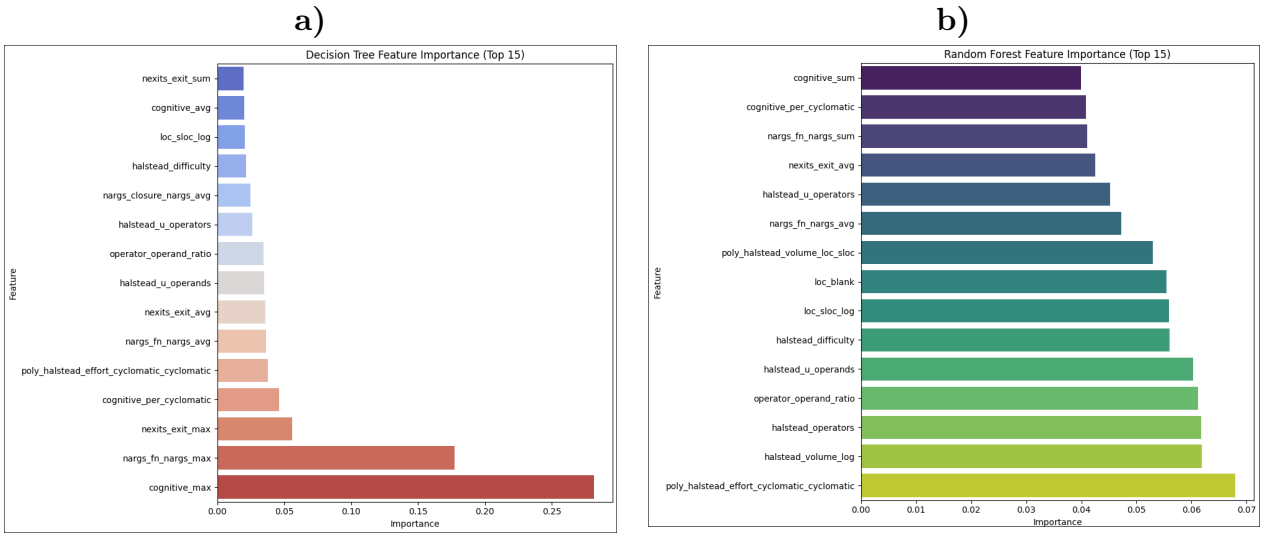

Figure 7: Confusion Matrices of predicted metrics: **a)** Decision Tree Feature Importance , **b)** Random Forest Feature Importance,

As shown in Figure 7, the Decision Tree model demonstrates a highly skewed importance distribution, with a sharp decline in predictive weight following the primary feature. This 'top-heavy' profile is characteristic of the Decision Tree's greedy splitting approach, where the root node (the most discriminative feature) captures the majority of the information gain. The model primarily relies on cognitive_max, followed closely by NARGS and NEXITS. This indicates that the tree prioritizes a mix of abstract logical complexity and concrete interface metrics when identifying architectural hubs. Interestingly, the raw metrics were prioritised for the Decision Tree over the polynomial metrics generated suggesting that structural base metrics provided a clearer picture for hub score rather than complex feature interactions. When evaluating the Random Forest model a very different distribution, as the ensemble nature allows many trees to have the same variables the distribution of values is closer together. Halstead metrics are by far the most popular metric used with 5 out of the first 6 including Halstead metrics. These metrics, which quantify program volume, vocabulary, and effort based on operator and operand counts, appear to provide the classifier with a more nuanced picture for identifying temporal hubs. This suggests that for the Random Forest, the density and variety of operations within a file are more indicative of evolutionary risk than solitary complexity or interface-level metrics

5.2.4 Extracted rules

The rules were extracted using a visualisation of the Decision Tree and using te2rules (Lal, Chen and Mithal, 2024) on the Random Forest.

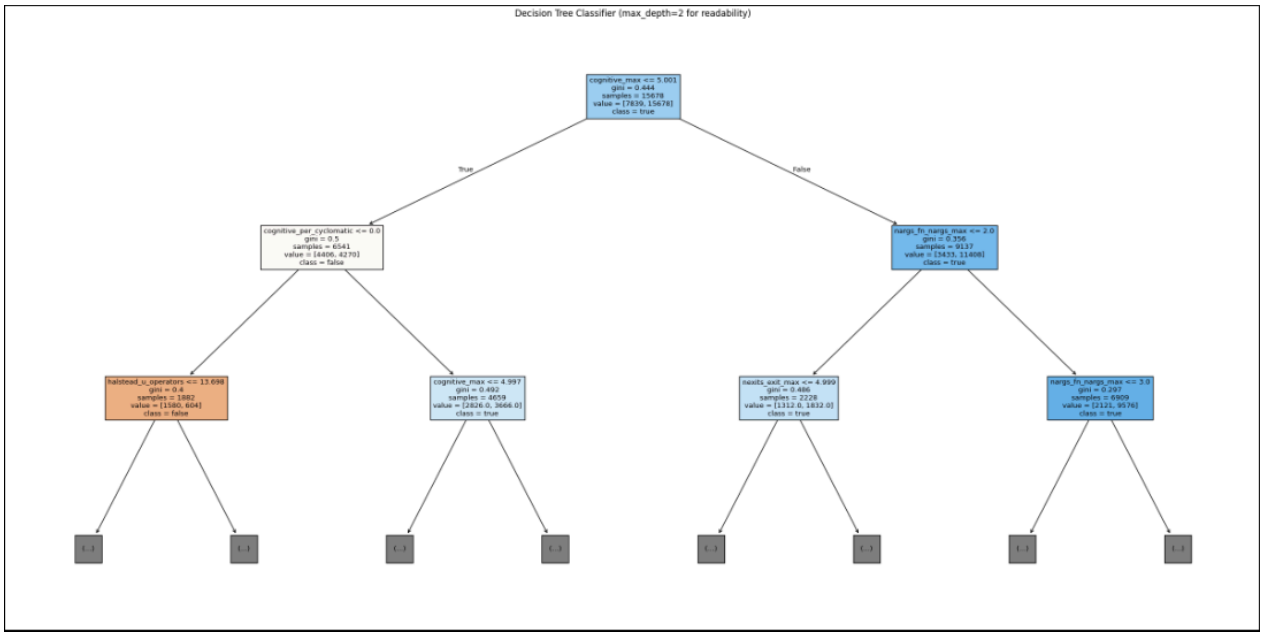

Figure 8: Decision Tree Visual representation

Figure 8 is a high level snapshot only showing the highest level of the 15 deep Decision Tree for readability. Revealing the simple rules that the decision tree uses such as cognitive_max <= 5 or cognitive_per_cyclomatic <= 0, this highlights the simplicity of the Decision Tree model at the cost of F1. A clear inverse relationship exists between the predictive power of Decision Trees and the performance of Random Forests. The rules generated by te2rules from the Random Forest were not actionable for humans, each of the top 5 most important rules derived by te2rules was at least 15 terms long making it entirely human unreadable. When coverage was evaluated for the top 5 performing rules the cumulative coverage was 0.27% of the dataset. This highlights that while interpretability was the focus the correlation between structural and temporal metrics is too weak to derive meaningful and interpretable rules.

5.2.5 External Validation

External validation was conducted on a suite of repositories from (*CERN/awesome-cern*, 2026) .These repositories were entirely excluded from the training phase to test the generalisability of the Random Forest model against Codescene's ground truth labels for 'Unhealthy' code (Health < 4).

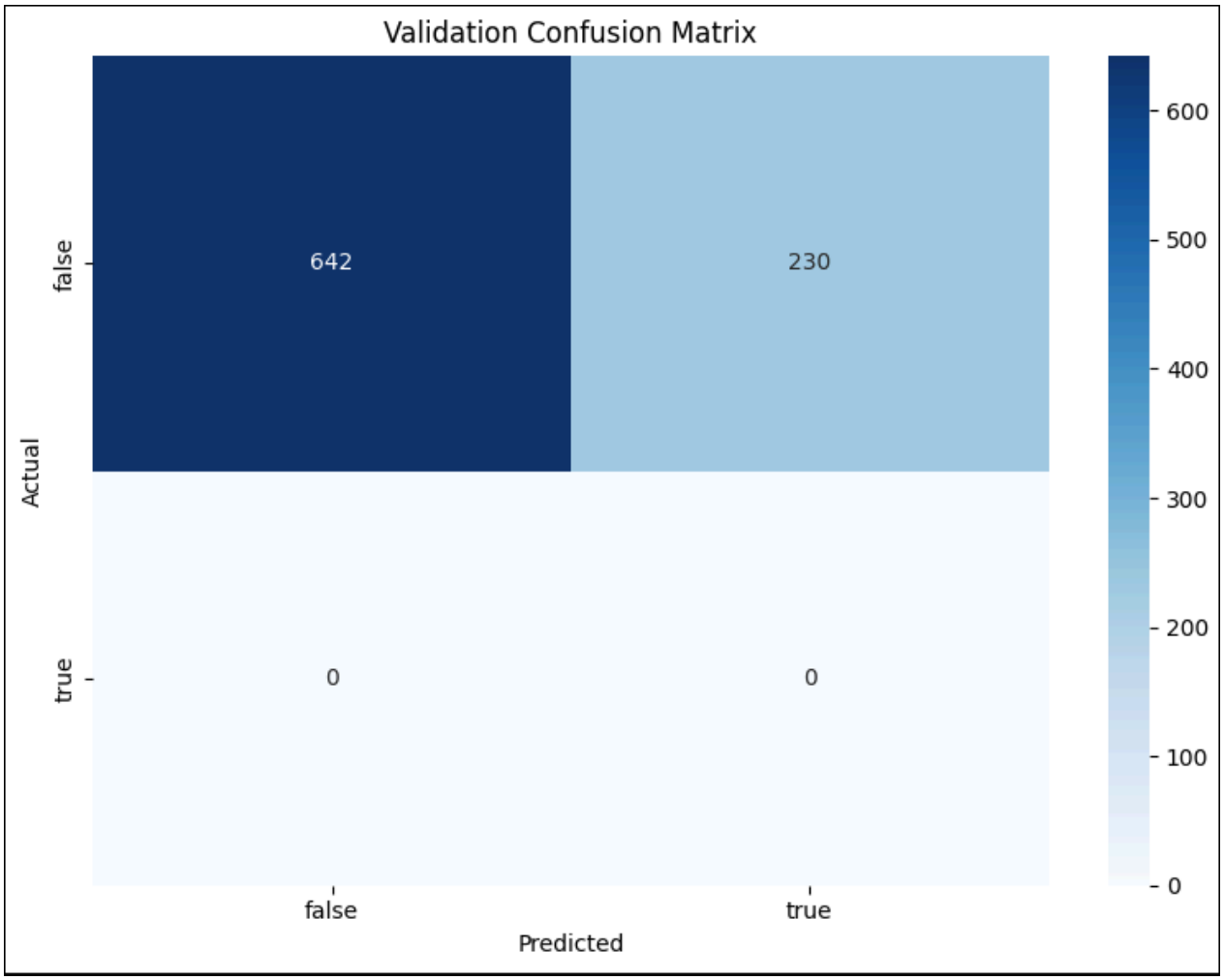

Figure 9: Confusion matrix of Random forest model on ACTS repository

Figure 9 reveals that the model demonstrated a strong predictive ability in line with the ground labels, although the Random Forest model had some false positives. This indicates that the Random Forest learned a partially generalisable relationship between structural metrics and extreme evolutionary risk to identify which files from ACTS are critically high risk.

5.3 Summary of Findings

5.3.1 Autonomous Labeling Accuracy (RQ1)

Using hub score a viable empirical alternative to the current Sota in expert rules (Codescene) was verified. Functioning as a high-pass filter, the Hub Score demonstrated a strong correlation with critical architectural vulnerabilities while effectively ignoring minor technical debt. This confirms that temporal metadata can autonomously label high-risk files, providing a scalable labeling mechanism for domain-specific maintainability research.

5.3.2 Opinionated Interpretable Guardrail Rule Derivation (RQ2)

Random forest with an F1 of (42%) outperformed Decision Trees (34%), the accuracy for both remained high due to the class imbalance in the final training set Figure 5 but both models failed on precision. This meant that many false positives were present, undermining the efficacy of the guardrail due to the uncertainty in predictions.

Efforts to derive rules from both the Decision Tree and Random Forest Model yielded mixed results. The Decision tree yielded a comprehensive flow of rules but the low predictive accuracy and tendency to over predict high risk rendered them unusable Figure 8. When evaluating the Random Forest model the derived rules were not usable not only due to their complexity but also the lack of coverage for practical application. Consequently, this study finds that while ML can predict historical maintainability risk to an extent, the extraction of simple, interpretable structural rules from those predictions remains a significant challenge.

6 Discussion

6.1 The "High-Pass" Nature of Evolutionary Risk (RQ1)

When examining the results from Table 2 it is clear that the hub score has a strong correlation with the critical risk files identified by codescene. The standard set out by (Borg, Ezzouhri and Tornhill, 2024) is too sensitive for hub score analysis. By isolating only the top 15% of files ($n = 9,000$) as seen in Figure 4, the metric focuses exclusively on the most critical architectural bottlenecks. This selectivity reduces the 'noise' associated with moderate technical debt, ensuring that when a file is labeled 'High Risk,' it represents a definitive priority for refactoring.

6.1.1 Temporal metrics as a Proxy for Structural Decay

As the hub score is entirely derived from temporal metrics, it reveals a correlation between historical metadata and the structural expert rules that are currently the standard across the field. Similarly to what (Faragó, Hegedűs and Ferenc, 2015) identified when observing the impact of code churn on maintainability,a composite representation of temporal metadata is shown to be correlated to the current industry standard structural metrics to an extent which cannot be overlooked. Historical metrics can and should be in tandem within static analysis to not only identify refactoring targets as codescene currently does. Both structural and temporal metrics should be used as a metric for identifying high risk files to supplement current refactoring methods by highlighting problems without a clear structural issue that would normally go undetected.

6.1.2 Implications for Large-Scale Research

This result validates the use of analytical methods as a scalable alternative manual expert defined rules. By circumventing the need for proprietary tooling or manual expert intervention, this methodology enables empirical studies of software health at a scale previously impossible in domain-specific areas.

6.2 The Structural-Temporal Mismatch (RQ2)

As observed in Table 4, a significant disparity exists between the high accuracy of the models and their relatively low F1 scores (34%–42%). This gap reveals that structural metrics, which provide a stateless snapshot of a file, are insufficient sole predictors of the stateful, temporal risk captured by the Hub Score.

The data suggests that code structure and evolution risk are often decoupled. For instance, a high-complexity "God file" may be a structural outlier but remain low-risk if it is so stable (or feared) that it is rarely modified. Conversely, a simple configuration file may exhibit high temporal risk simply because it is updated in every version. These counterexamples demonstrate that a file's structural "badness" is not a reliable proxy for its status as a historical hotspot.

Logically this mismatch is expected, there may be a weak correlation between temporal changes and structural metrics but there are too many examples which serve as counterexamples to this being a strong correlation: A god file so large people are afraid to interact with in new changes being labelled as low risk by hub score and high risk structurally, a config file that changes every version labelled as high risk by hub score and low risk structurally etc.

The strides made by (Xiao *et al.*, 2016) that temporal analysis alone can identify bug-prone files, the results here indicate that a hybrid approach is necessary one where temporal and structural metrics are used together. Moving forward, the focus on file level metrics for prediction should move to a platform metric position, identifying critical risk files by assessing its place in the overall repository is a clear path forward as this study has show that within a singular file there is not enough of a correlation between a badly structured file and a historical hotspot for the ML models to be able to accurately predict.

6.2.1 Confidence in Actionable Guardrails

The choice in using file level metrics was made to facilitate the generation of human-readable, interpretable rules. However, the results suggest a fundamental disconnect between machine-learned patterns and actionable developer guardrails.

The core idea of the interpretable rules using only structural metrics was to allow for refactoring goals to be created with those in mind. High dimensional structural metrics such as cognitive_max or poly_halstead_effort_cyclocmatic_complexity provide the models with the tools to reach their current F1 score. They are fundamentally different to how expert rules are created to target areas for refactoring. The rules derived from these models were either too granular (overfitted to specific files) or relied on abstract composite metrics that offer little guidance for refactoring.

For rules to be actionable it must provide a clear target a developer can understand and fix. As the machine learning models prioritise statistical probability over developer comprehensibility, the resulting lack of rules clarity is a side effect of attempting to derive comprehensible rules from statistical machines.

6.3 What this means for Research Software

6.3.1 The Empirical Ground Truth

The capabilities of hub score file classification mark a path for researchers to conduct empirical analysis on code maintainability without relying on preestablished generalised expert rules or hand annotating data. Allowing researchers to quickly and efficiently analyse large amounts of repositories to conduct domain specific or large scale analysis with minimal compute. Providing a standardised marker gathered from targeted analysis to serve as a foundation for future studies

6.3.2 The changing landscape from Humans to AI

The focus on ensuring the models could generate human interpretable rules through the use of Decision Trees and Random Forests limited the capabilities of the models used. The current research landscape will begin using generative AI more in the same way the entire software engineering industry is moving. If that is the case, should the focus be on the understandable and interpretable refactoring rules? Should researchers spend time, effort and money on learning how to write software to the same standard as a full time software engineer? Researchers should be able to focus on their research and tools should guarantee the floor of software quality automatically to avoid issues such as (Trisovic *et al.*, 2022) ensuring that research can be used, extended and verified post publishing.

6.3.3 Analytical Black Boxes

This study posits that a black box approach using a model such as a Convolutional Neural Network is the path forward for machine learning maintainability detection. These models cannot produce human interpretable rules but they can with a high accuracy predict classes for analysed data. A blanket high or low risk assessment would be an inefficient use of time and resources for hand writing code; a generated machine learning model could quickly rewrite a file based on a high risk flag without the same time commitment it would take a human. This approach would ensure that the inefficient code patterns produced by AI noted by (Harding, 2025) are caught before even being presented to the user by running the maintainability analysis alongside the code generation tooling. A focus on recall is to ensure all bad code gets flagged, even if 'ghost echoes' still appear a generative AI agent can rewrite the code quickly and efficiently to hit the criteria set out by the black box method. This is a proposed approach for raising the code quality of research software in a sustainable effortless way.

6.4 Threats to Validity

6.4.1 Construct Validity

Construct validity examines whether the theoretical constructs (e.g., "Architectural Decay") are accurately measured by the chosen metrics (the Hub Score).

- Temporal Proxy Limitations: While the hub score has been observed to act as a high pass filter for historical hotspots, it can only act as a proxy for measurement. This is because a file can exhibit high churn due to non architectural reasons. The composite nature of Hub score does mitigate this; there is still a risk for files marked as high-risk to be low risk and low risk files being misclassified as high risk by Hub score.
- Expert Rule Alignment: This study has argued that domains require specific rules while simultaneously using expert derived universal rules as "ground truth". This study argues that Hub Score is a more empirical measurement; it is worth noting that without an agreed upon method of measuring maintainability, hub score is only one of many approaches for measuring maintainability.

6.4.2 Internal Validity

Internal validity relates to factors within the experimental design that may have biased the results.

- Computational Constraints: Due to the use of a single execution environment the analysis of commit history was capped in both commit count and number of changes per commit to ensure feasible processing times. This may have led to "deep history" of older repositories being missed that could've influenced the overall hub score distribution.

6.4.3 External Validity

External validity concerns the generalisability of the findings to other contexts.

- Case Study Comparison: Due to constraints of using the Codescene client, large scale analysis of repositories was unavailable, this led to a case study selection to be used for external verification. This sample followed the same criterion as the repositories in the dataset but may yet have been biased for or against the study.

6.5 Future Work

Future work relates to improvements in approach and method that could be implemented to progress this project.

- Historical file analysis: The method in which this project retrieved files to analyse was only retrieving files from the HEAD of each repository, a method to retrieve and label the deleted files could lead to a richer dataset with deleted being a metric to inform hub score accuracy as well as increasing sample size.
- New Domain Application: This study was constrained to C++ on research repositories alone, a meta analysis across domains and languages could identify if temporal patterns are language and domain agnostic.
- Metric extension: This study used only file level metrics as input for machine learning models, a platform level view or class level metrics could yield better results in predictive analysis where applicable.
- Black box model: As discussed a black box model using non interpretable rules within an AI generation loop could yield very potent results in raising the floor on research software quality.

7 Conclusion

This study sought to address the growing challenge of structural decay in research software by developing an opinionated framework of "Architectural Guardrails". By leveraging machine learning to derive these rules, this study established an empirical, domain-specific truth that departs from traditional, domain-agnostic industry standards.

Hub score was introduced as a method to label data for maintainability risk at scale giving the ability to automatically create large high quality domain specific maintainability datasets without leaning on predetermined expert rules or hand annotation. Giving a standardised approach to create domain specific maintainability datasets for future research easily. The findings demonstrated when compared to current state of the art tooling (Codescene) that hub score has comparable results when only using temporal metrics as when expert rules are used to identify critical risk files. Suggesting that maintainability detection should not only use temporal data as signals to look for structural issues but temporal data should be used as a standalone diagnostic for flagging risk within files without measurable structural issues as refactoring targets.

The machine learning models trained on the file level metrics of 200+ repositories yielded mixed results. The overall F1 score of both the Decision Tree and Random Forest models were too low to be of any value for deriving opinionated guardrails as the false positive rate was too high to be trusted to be the single source of truth on maintainability problems within a research repository. Interpretable rules were a goal of the models which served as a limitation that could not be overcome with the dataset. Focusing on attempting to generate interpretable rules created is not a feasible approach when using a machine learning method based on structural metrics. To provide the granularity for these models to have the chance to generate accurate predictions incomprehensible polynomial metrics are required defeating the purpose of using an interpretable model.

Research software is and always has been at the forefront of innovation, the scientific community needs software to progress. Yet, it currently requires domain experts in fields such as medicine, physics and psychology to develop software engineering skills on par with software engineers to develop clean, maintainable verifiable code. The long-term solution lies in sustaining the momentum of the Research Software Engineering (RSE) movement, recognizing these experts as critical stakeholders in the research lifecycle, from funding to publication. Strides have been made to progress this initiative but until that goal is achieved we cannot neglect research software quality, the floor of research software quality has to be raised to give confidence to results being published, to allow researchers to build upon the work of each other.

As every facet of software moves closer to AI, to achieve sustainable programming standards the focus must shift from human-centric linting to AI-driven automation. Utilising analytical black box models such as CNNs to flag high risk files within the generation loop requiring the agents to resolve the problems before even presenting them to the researcher is an effortless way to raise the software floor quality without burdening

researchers. This study proposes that through the use of hub score annotated data alongside the current sota structural analysis of files a model could be trained to achieve this. This paradigm shift ensures that research software remains a robust foundation for scientific discovery, verification, and extension in the decades to come.

Bibliography

Ardito, L. *et al.* (2020) "rust-code-analysis: A Rust library to analyze and extract maintainability information from source codes," *SoftwareX*, 12, p. 100635. Available at: https://doi.org/10.1016/j.softx.2020.100635.

Bakota, T. *et al.* (2011) "A probabilistic software quality model," in, pp. 243–252. Available at: https://doi.org/10.1109/ICSM.2011.6080791.

Bertrand, S. *et al.* (2023) "Replication and Extension of Schnappinger's Study on Human-level Ordinal Maintainability Prediction Based on Static Code Metrics," *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, pp. 241–246. Available at: https://doi.org/10.1145/3593434.3593488.

Bertrand, S., Favier, P.-A. and Andre, j.-m. (2022) "Building an Operable Graph Representation of a Java Program as a Basis for Automatic Software Maintainability Analysis," in, pp. 243–248. Available at: https://doi.org/10.1145/3530019.3534081.

Bluemke, I. and Borsukiewicz, P. (2023) "Experiments on software error prediction using Decision Tree and Random Forest algorithms", pp. 865–869. Available at: https://doi.org/10.15439/2023F363.

Boetticher, G., Menzies, T. and Ostrand, T. (2007) "PROMISE Repository of empirical software engineering data."

Borg, M., Ezzouhri, M. and Tornhill, A. (2024) *Ghost Echoes Revealed: Benchmarking Maintainability Metrics and Machine Learning Predictions Against Human Assessments.* arXiv. Available at: https://doi.org/10.48550/arXiv.2408.10754.

Breiman, L. (2001) "Random Forests," *Machine Learning*, 45(1), pp. 5–32. Available at: https://doi.org/10.1023/A:1010933404324.

Cai, Y. *et al.* (2025) "Understanding Architectural Complexity, Maintenance Burden, and Developer Sentiment -A Large-Scale Study," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pp. 2176–2187. Available at: https://doi.org/10.1109/ICSE55347.2025.00168.

CERN/awesome-cern (2026). European Organization for Nuclear Research. Available at: https://github.com/CERN/awesome-cern (Accessed: April 22, 2026).

Chawla, N.V. *et al.* (2002) "SMOTE: Synthetic Minority Over-sampling Technique," *Journal Of Artificial Intelligence Research* [Preprint]. Available at: https://doi.org/10.1613/jair.953.

Coleman, D. *et al.* (1994) "Using Metrics to Evaluate Software System Maintainability," *Computer*, 27, pp. 44–49. Available at: https://doi.org/10.1109/2.303623.

Faragó, C., Hegedűs, P. and Ferenc, R. (2015) "Cumulative code churn: Impact on maintainability," in *2015 IEEE 15th International Working Conference on Source Code*

Analysis and Manipulation (SCAM), pp. 141–150. Available at: https://doi.org/10.1109/SCAM.2015.7335410.

Goble, C. (2014) "Better Software, Better Research," *IEEE Internet Computing*, 18(5), pp. 4–8. Available at: https://doi.org/10.1109/MIC.2014.88.

Gosain, A. and Sharma, G. (2015) "Static Analysis: A Survey of Techniques and Tools," in D. Mandal et al. (eds.) *Intelligent Computing and Applications*. New Delhi: Springer India, pp. 581–591. Available at: https://doi.org/10.1007/978-81-322-2268-2_59.

Haddouchi, M. and Berrado, A. (2025) "Forest-ORE: Mining an optimal rule ensemble to interpret random forest models," *Engineering Applications of Artificial Intelligence*, 143, p. 109997. Available at: https://doi.org/10.1016/j.engappai.2024.109997.

Hannay, J.E. *et al.* (2009) "How do scientists develop and use scientific software?," in *2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, pp. 1–8. Available at: https://doi.org/10.1109/SECSE.2009.5069155.

Harding, W. (2025) *AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones - GitClear*. Available at: https://www.gitclear.com/ai_assistant_code_quality_2025_research (Accessed: April 28, 2026).

Howison, J. and Herbsleb, J.D. (2014) "The sustainability of scientific software:," *2014* [Preprint]. Available at: https://doi.org/https://james.howison.name/pubs/HowisonHerbsleb-Sustainability.pdf.

ISO 25010 (no date). Available at: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010 (Accessed: February 2, 2026).

Jenks, G.F. (1967) "The data model concept in statistical mapping," *International Yearbook of Cartography*, 7, pp. 186–190. Available at: https://cir.nii.ac.jp/crid/1573668925394541312.

Jiménez, R.C. *et al.* (2017) *Four simple recommendations to encourage best practices in research software*. F1000Research. Available at: https://doi.org/10.12688/f1000research.11407.1.

Johnson, B. *et al.* (2013) "Why don't software developers use static analysis tools to find bugs?," in *2013 35th International Conference on Software Engineering (ICSE)*, pp. 672–681. Available at: https://doi.org/10.1109/ICSE.2013.6606613.

Johnson, S. and Hill, M. (1978) "Lint, a C Program Checker," in. Available at: https://www.semanticscholar.org/paper/Lint%2C-a-C-Program-Checker-Johnson-Hill/74617cffa3c6438d04aa99bef1cca415de47d0d3 (Accessed: February 8, 2026).

Journal of Open Source Software (no date). Available at: https://joss.theoj.org/ (Accessed: April 2, 2026).

Khoshgoftaar, T.M. and Allen, E.B. (2001) “Controlling Overfitting in Classification-Tree Models of Software Quality,” *Empirical Software Engineering*, 6(1), pp. 59–79. Available at: https://doi.org/10.1023/A:1009803004576.

Lal, G.R., Chen, X. and Mithal, V. (2024) *TE2Rules: Explaining Tree Ensembles using Rules*. Available at: https://arxiv.org/abs/2206.14359.

Marwick, B. (2017) “Computational Reproducibility in Archaeological Research: Basic Principles and a Case Study of Their Implementation,” *Journal of Archaeological Method and Theory*, 24(2), pp. 424–450. Available at: https://doi.org/10.1007/s10816-015-9272-9.

Meyer, B. (2009) *Object-oriented software construction*. 2. ed., 16. print. Upper Saddle River, NJ: Prentice Hall PTR.

Oman, P. and Hagemeister, J. (1992) “Metrics for assessing a software system's maintainability,” in *Proceedings Conference on Software Maintenance 1992*, pp. 337–344. Available at: https://doi.org/10.1109/ICSM.1992.242525.

Parnas, D.L. (1972) “On the criteria to be used in decomposing systems into modules,” *Commun. ACM*, 15(12), pp. 1053–1058. Available at: https://doi.org/10.1145/361598.361623.

Pedregosa, F. *et al.* (2011) “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Quinlan, J.R. (1993) *C4.5: programs for machine learning*. San Mateo, Calif: Morgan Kaufmann Publishers (The Morgan Kaufmann series in machine learning).

Schnappinger, M., Fietzke, A. and Pretschner, A. (2020) “Defining a Software Maintainability Dataset: Collecting, Aggregating and Analysing Expert Evaluations of Software Maintainability,” in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 278–289. Available at: https://doi.org/10.1109/ICSME46990.2020.00035.

Schnappinger, M. *et al.* (2019) “Learning a classifier for prediction of maintainability based on static analysis tools,” in *Proceedings of the 27th International Conference on Program Comprehension*. Montreal, Quebec, Canada: IEEE Press (ICPC '19), pp. 243–248. Available at: https://doi.org/10.1109/ICPC.2019.00043.

team, T.p.d. (2020) *pandas-dev/pandas: Pandas*. Zenodo. Available at: https://doi.org/10.5281/zenodo.3509134.

Technology, I. (2015) “Neo4j, the World's Leading Graph Database,” *Neo4j Graph Database* [Preprint].

Trisovic, A. *et al.* (2022) “A large-scale study on research code quality and execution,” *Scientific Data*, 9, p. 60. Available at: https://doi.org/10.1038/s41597-022-01143-6.

Weyuker, E.J. (2011) "Empirical Software Engineering Research - The Good, The Bad, The Ugly," in *2011 International Symposium on Empirical Software Engineering and Measurement*, pp. 1–9. Available at: https://doi.org/10.1109/ESEM.2011.66.

Wilson, G. (2006) "Software Carpentry: Getting Scientists to Write Better Code by Making Them More Productive," *Computing in Science & Engineering*, 8(6), pp. 66–69. Available at: https://doi.org/10.1109/MCSE.2006.122.

Wilson, G. *et al.* (2017) "Good enough practices in scientific computing," *PLOS Computational Biology*, 13(6), pp. 1–20. Available at: https://doi.org/10.1371/journal.pcbi.1005510.

Xiao, L. *et al.* (2016) "Identifying and quantifying architectural debt," in *Proceedings of the 38th International Conference on Software Engineering*. Austin Texas: ACM, pp. 488–498. Available at: https://doi.org/10.1145/2884781.2884822.